

Article

Robust Quadrotor Trajectory Tracking Under Multimodal Wind Disturbances via Residual-Aware Deep Reinforcement Learning

Kunpeng Qi , Chunhong Liu and Zhihong Liu * 

College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, China; qkp24@nudt.edu.cn (K.Q.); liuchunhong23@nudt.edu.cn (C.L.)

* Correspondence: zhliu@nudt.edu.cn

Highlights

What are the main findings?

- This paper proposes a residual-aware deep reinforcement learning framework for robust quadrotor trajectory tracking under multimodal wind disturbances, which combines a compact acceleration-residual feature, residual-input regularization, and a tail-risk-aware reward function to improve both average tracking accuracy and tail-error robustness.
- Extensive experiments under multimodal in-distribution wind, held-out out-of-distribution wind, and measured real-wind disturbances confirm the proposed method's robustness superiority.

What are the implications of the main findings?

- Compact residual awareness can provide disturbance-related information without requiring an explicit wind sensor, a full disturbance observer, or a long-history recurrent estimator, while residual-input regularization can limit excessive policy dependence on residual patterns specific to the training wind distribution and improve out-of-distribution robustness.
- A tail-risk-aware reward function can jointly promote tracking accuracy, control smoothness, and suppression of large tracking deviations.

Abstract

Robust quadrotor trajectory tracking under wind disturbances is challenging because real outdoor wind is multimodal with time-varying and heavy-tailed characteristics, whereas existing solutions suffer from insufficient disturbance observability and poor out-of-distribution robustness. To this end, this paper presents a robust quadrotor trajectory-tracking method based on residual-aware deep reinforcement learning for multimodal wind disturbances. The proposed method augments a standard tracking policy with a compact online acceleration-residual feature, which provides disturbance-related information without requiring an explicit wind sensor, a full disturbance observer, or a long-history recurrent estimator. To reduce excessive dependence on wind-specific temporal patterns, a residual-input regularization term is introduced during policy optimization. In addition, a tail-risk-aware reward is designed to balance nominal tracking accuracy, control smoothness, and suppression of large tracking deviations. The proposed method is evaluated under in-distribution wind, held-out out-of-distribution wind, and measured real-wind disturbances. The results show that the proposed method achieves the most balanced robustness under multimodal wind conditions compared with baselines.



Academic Editor: Octavio Garcia-Salazar

Received: 21 June 2026

Revised: 22 July 2026

Accepted: 24 July 2026

Published: 27 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)

[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: quadrotor; trajectory tracking; deep reinforcement learning; multimodal wind disturbance; disturbance residual observation; robust control

1. Introduction

Multicopter unmanned aerial vehicles (UAVs) are now widely used in inspection, environmental monitoring, search and rescue, precision agriculture, and low-altitude logistics [1]. Among these platforms, quadrotors are particularly attractive because of their vertical take-off and landing capability, hovering ability, and agile maneuverability. In many outdoor missions, accurate trajectory tracking is a fundamental requirement. However, outdoor flight is inevitably affected by wind, which can directly degrade tracking accuracy and even cause mission failure. More importantly, real wind is rarely a stationary or unimodal disturbance. It may contain slowly varying background flow, periodic components, intermittent gusts, step-like variations, Gaussian-like fluctuations, and heavy-tailed turbulent residuals. In this paper, disturbances generated by these coupled components are referred to as multimodal wind disturbances with time-varying and heavy-tailed characteristics. Therefore, robust trajectory tracking under such multimodal wind disturbances remains a practically important and technically challenging problem.

Model-based control has long been a common route to this problem. Differential-flatness-based minimum-snap trajectory generation and geometric tracking control on the special Euclidean group $SE(3)$ provide a strong foundation for aggressive flight [2,3], and the underlying rigid-body model is well established [4]. To handle uncertainty, robust and disturbance-rejection controllers extend this foundation. Representative examples include PID control and its variants [5], game-theoretic robust tracking that treats wind as an adversarial input [6], and adaptive finite-time backstepping with disturbance estimation [7]. These methods perform well when the disturbance stays within an assumed class. Their accuracy, however, depends on the fidelity of the dynamics model and on parameter identification. Under multimodal wind, fixed assumptions may require conservative margins that reduce tracking aggressiveness.

Another line of work explicitly models and compensates the disturbance. Disturbance observers, active disturbance rejection control, and active wind rejection schemes estimate the lumped disturbance online and cancel it inside the feedback loop [8–10]. Data-driven studies go further and learn the unmodeled part of dynamics. Hybrid aerodynamic models and data-driven model predictive control improve prediction when aerodynamic effects matter [11,12], and recent diffusion-based dynamics learning suggests that quadrotor residual dynamics are genuinely multimodal rather than unimodal [13]. These studies show that residual dynamics are informative and can exhibit a multimodal structure. However, they usually require a separate estimator or dynamics model, which requires additional design effort and computational cost.

In addition, deep reinforcement learning (DRL) offers a complementary, data-driven route by optimizing feedback policies through interaction with a simulated environment. In quadrotor control, DRL has been applied to attitude stabilization, controller-gain prediction, adaptive trajectory tracking, fast end-to-end control, and wind disturbance rejection [14–17]. Recent studies have also reported expert-level performance in drone racing and high-speed flight [18–20]. Despite these advances, robust DRL control under multimodal wind remains difficult. The performance of a learned policy strongly depends on the disturbance distribution used during training and on how the policy represents external disturbances. When the wind is multimodal with time-varying or heavy-tailed characteristics, simply enlarging the randomization range can make policy

optimization harder. It can also push the policy toward a conservative compromise across disturbance modes, which may lower training efficiency and reduce tracking accuracy in extreme cases. Moreover, if the observation contains no quantity related to the external disturbance, the policy can only infer the disturbance indirectly from position and velocity errors. This indirect inference is sensitive to model uncertainty, actuator lag, and sensor noise. A more effective design should therefore reduce, as much as practical, the policy's dependence on specific disturbance patterns.

Furthermore, risk sensitivity is another important issue. For learning-based methods, standard tracking rewards usually emphasize mean tracking accuracy and control effort. However, outdoor wind disturbances often produce rare but severe deviations, and a controller with low average error may still be unsafe if its tail errors are large. Conditional value-at-risk (CVaR) and related risk-sensitive objectives provide a principled way to penalize the tail of the error distribution [21–23]. For quadrotor trajectory tracking, such tail-risk-aware learning can encourage the policy to suppress large deviations rather than optimize only average performance.

Robustness and generalization remain central concerns for learning-based control. Proximal policy optimization (PPO) is a practical optimizer for such continuous-action tasks because its clipped objective supports stable on-policy updates [24], and a recent study has compared learned and optimal control on the same racing task [25]. The robustness of learned quadrotor controllers across changing operating conditions depends strongly on the training distribution, observation design, actuation model, and evaluation protocol [26], while domain randomization is a common way to expose the policy to diverse simulated conditions [27]. A further concern is shortcut learning, in which a policy relies on features that are predictive during training but not causally reliable under distribution shift [28]. In a disturbance-rich setting, a long residual history may encode wind-specific temporal fingerprints. This work therefore combines compact residual observation, residual-input regularization, and a CVaR-style tail-risk penalty to retain useful disturbance information while limiting such shortcuts.

Motivated by this, this paper proposes a robust quadrotor trajectory-tracking method based on residual-aware deep reinforcement learning for multimodal wind disturbances. The key idea is to use a compact online acceleration residual as a lightweight disturbance-related observation. The residual is computed as the mismatch between measured acceleration and the theoretical acceleration computed from the thrust and gravity terms. This feature improves disturbance awareness without requiring an explicit wind sensor, a full disturbance observer, or a long-history recurrent estimator. To prevent excessive dependence on wind-specific residual patterns, a residual-input regularization term is added to limit the sensitivity of the policy to residual noise. To further improve robustness against extreme disturbances, a tail-risk-aware reward is designed by combining position tracking, action smoothness, acceleration smoothness, and a CVaR-style penalty, thereby balancing nominal tracking accuracy, control smoothness, and suppression of large tracking deviations.

The main contributions of this paper are summarized as follows:

- A residual-aware DRL trajectory-tracking framework is proposed by augmenting a standard policy with a compact acceleration-residual feature, residual-input regularization, and a tail-risk-aware reward function, which improves both average tracking accuracy and tail-error robustness.
- A multimodal wind disturbance pool is constructed by combining correlated stochastic wind, periodic components, intermittent gusts, step-like changes, Gaussian-like baseline perturbations, and heavy-tailed residual disturbances. This design

gives the policy a more realistic and diverse disturbance distribution than a single Gaussian perturbation.

- Extensive experiments are conducted under in-distribution, out-of-distribution, and measured real-wind disturbances. The results show that the proposed method achieves the most balanced robustness under multimodal wind conditions compared with baselines.

The remainder of this paper is organized as follows. Section 2 introduces the quadrotor dynamics and the problem description. Section 3 presents the proposed methodology. Section 4 describes the implementation, including the low-level control interface and the control allocation. Section 5 reports the experimental results and discussion. Section 6 concludes the paper.

2. Preliminaries

2.1. Quadrotor Dynamics Model

The quadrotor is modeled as a rigid body with six-degree-of-freedom translational and rotational dynamics. Let $\mathbf{p} \in \mathbb{R}^3$, $\mathbf{v} \in \mathbb{R}^3$, $\mathbf{R} \in \text{SO}(3)$, and $\boldsymbol{\omega} \in \mathbb{R}^3$ denote the world-frame (inertial-frame) position, world-frame linear velocity, attitude rotation matrix, and body-frame angular velocity, respectively, where $\text{SO}(3)$ is the special orthogonal group of three-dimensional rotation matrices and $\text{SE}(3) = \mathbb{R}^3 \times \text{SO}(3)$ is the special Euclidean group of rigid-body poses. The continuous-time dynamics follow the standard rigid-body quadrotor model [3,4], and are given in (1)–(4). In the simulation, the inertial-frame position and velocity are obtained directly from the physics engine as ground-truth states, emulating a perfectly fused state estimate:

$$\dot{\mathbf{p}} = \mathbf{v}, \quad (1)$$

$$m\dot{\mathbf{v}} = \mathbf{R}\mathbf{f}^B + m\mathbf{g} + \mathbf{f}_w + \mathbf{f}_{\text{aero}}, \quad (2)$$

$$\dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}]_{\times}, \quad (3)$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} + \boldsymbol{\tau}_w, \quad (4)$$

where m is the vehicle mass, $\mathbf{J} \in \mathbb{R}^{3 \times 3}$ is the inertia matrix, $\mathbf{g} = (0, 0, -g_0)^\top$ is the gravity vector, and $[\boldsymbol{\omega}]_{\times}$ denotes the skew-symmetric matrix associated with $\boldsymbol{\omega}$. The vector $\mathbf{f}^B$ is the body-frame collective thrust produced by the rotors, and $\boldsymbol{\tau}$ is the body-frame control torque. The terms $\mathbf{f}_w$ and $\boldsymbol{\tau}_w$ represent the wind disturbance force and torque, and $\mathbf{f}_{\text{aero}}$ represents aerodynamic residual forces such as rotor drag and body drag. In the implementation, the policy action is not a direct motor command. It passes through a low-level rate-and-thrust control interface that maps the command into rotor forces before the physics engine integrates the dynamics. This setup follows the practical design of learning-based quadrotor controllers, where the RL policy handles high-level stabilization and tracking while the low-level interface handles fast actuator dynamics.

2.2. Problem Description

The control problem considered in this paper is robust quadrotor trajectory tracking under multimodal wind disturbances. A reference trajectory $\mathbf{p}^{\text{ref}} : [0, T] \rightarrow \mathbb{R}^3$ is given over a finite horizon T . At each control step t , the quadrotor state $\mathbf{x}_t = (\mathbf{p}_t, \mathbf{v}_t, \mathbf{R}_t, \boldsymbol{\omega}_t)$ evolves according to the dynamics in (1)–(4) under an external wind force $\mathbf{f}_{w,t}$ that depends on the realized wind vector $\mathbf{w}_t \in \mathbb{R}^3$. The instantaneous Euclidean tracking error is defined in (5) as

$$e_t = \|\mathbf{p}_t - \mathbf{p}_t^{\text{ref}}\|_2. \quad (5)$$

The wind sequence is treated as a random process. During training, the wind sequence $\mathbf{w}_{0:T-1}$ is drawn from a multimodal training distribution $\mathcal{P}_w^{\text{train}}$, whose explicit construction is given in Section 3.2. The trajectory $\mathbf{p}^{\text{ref}}$ is drawn from a trajectory distribution $\mathcal{P}_{\text{ref}}$. The control objective is to find a feedback policy π_θ , parameterized by θ , that maps the policy observation $\mathbf{o}_t$ to a normalized command $\mathbf{u}_t$ so that the expected discounted tracking cost is minimized. This optimization problem is written in (6):

$$\min_{\theta} \mathbb{E}_{\mathbf{p}^{\text{ref}} \sim \mathcal{P}_{\text{ref}}, \mathbf{w}_{0:T-1} \sim \mathcal{P}_w^{\text{train}}} \left[\sum_{t=0}^{T-1} \gamma^t \downarrow(\mathbf{p}_t, \mathbf{p}_t^{\text{ref}}, \mathbf{u}_t) \right], \quad (6)$$

subject to the dynamics in (1)–(4), where $0 < \gamma \leq 1$ is the discount factor and $\downarrow$ is a per-step cost that abstracts tracking error and control effort. The specific instantiation of the per-step cost $\downarrow$, which combines position tracking, motion smoothness, and a CVaR-style tail-risk penalty, is detailed in Section 3.3.4. Robust tracking is then expressed as a generalization requirement on the same policy: when the deployment wind sequence is drawn from a shifted distribution $\mathcal{P}_w^{\text{test}} \neq \mathcal{P}_w^{\text{train}}$, or from a measured real-wind process, the policy should still keep both the mean and the upper tail of the tracking error small over the evaluation trajectories. This requirement is captured by the robustness criterion in (7):

$$\mathcal{J}(\pi_\theta; \mathcal{P}_w^{\text{test}}) = \mathbb{E} \left[\frac{1}{N} \sum_t e_t \right] + \beta \text{CVaR}_\alpha(e_t), \quad (7)$$

where $\text{CVaR}_\alpha(\cdot)$ is the conditional value-at-risk at level α , and $\beta \geq 0$ trades off the mean error against the tail error. The methodology in Section 3 instantiates this criterion with a CVaR-style tail-risk penalty and a residual-aware policy. Section 5 reports the corresponding experimental results under in-distribution, out-of-distribution, and measured real-wind disturbances.

3. Methodology

3.1. Framework Overview

Figure 1 summarizes the proposed framework. The proposed method augments a standard quadrotor trajectory-tracking RL task with several coupled modules, including a multimodal wind disturbance pool, a PPO-style trajectory-tracking controller, an online acceleration-residual observation channel, and a disturbance-rejection reward. The wind disturbance pool is injected into the physics simulator as an external force applied to the quadrotor body. The policy receives the conventional tracking observation together with a compact residual feature that summarizes recent unmodeled acceleration. The actor outputs a normalized command, which is transformed by the low-level rate-and-thrust interface, while the critic evaluates rollouts from the global state. The optimizer follows a standard PPO update using the disturbance-rejection reward. The implementation is modular: configuration flags switch the same framework among a baseline policy without residual input, a compact residual-aware policy, and longer-window residual variants for sensitivity analysis. After training, the learned policies are evaluated under in-distribution, out-of-distribution, and measured real-wind disturbances, using tracking-error and robustness metrics including average tracking error, E_{95} error, $\text{CVaR}_{0.9}$ error, and progress completion ratio.

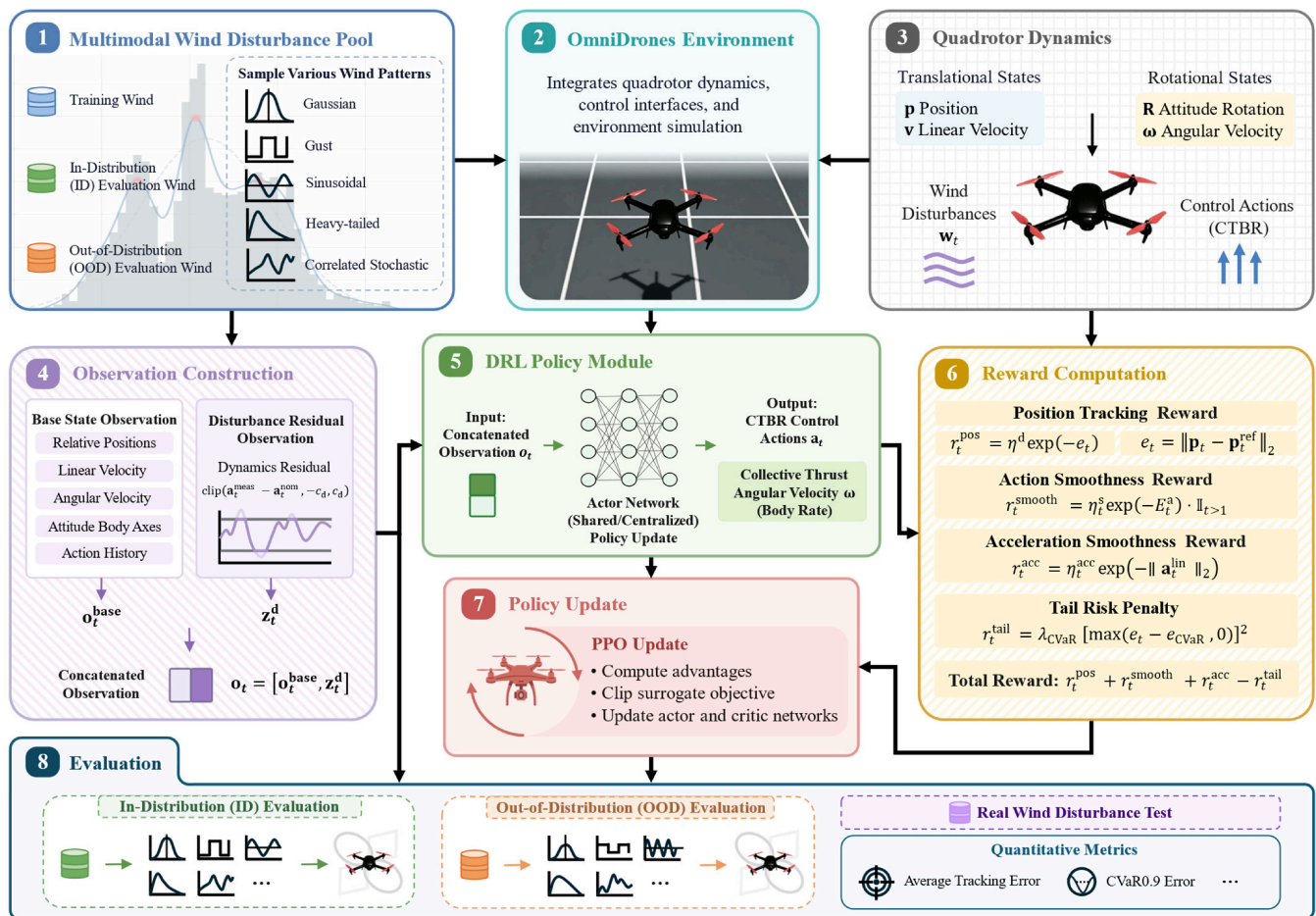


Figure 1. Overall framework of the proposed method.

3.2. Multimodal Wind Disturbance Model

The central design of the proposed method is a wind disturbance pool that is more diverse than a single Gaussian perturbation. Each wind realization is a three-dimensional time series obtained by combining temporally correlated base wind, periodic components, intermittent gusts, step-like variations, and heavy-tailed residual modulation. At each control step, the wind vector is decomposed as in (8):

$$\mathbf{w}_t = \mathbf{w}_t^{\text{base}} + \mathbf{w}_t^{\text{per}} + \mathbf{w}_t^{\text{gust}} + \mathbf{w}_t^{\text{tail}}, \quad (8)$$

where $\mathbf{w}_t^{\text{base}}$ carries colored stochastic wind together with slow correlated variations; $\mathbf{w}_t^{\text{per}}$ contains sinusoidal components; $\mathbf{w}_t^{\text{gust}}$ represents intermittent gust events with random direction, duration, and decay; and $\mathbf{w}_t^{\text{tail}}$ represents heavy-tailed residual modulation. At a distributional level, the wind generator can be summarized as a mixture of disturbance modes in (9):

$$\mathbf{w}_t \sim \sum_{k=1}^K \pi_k \mathcal{D}_k(\theta_k), \quad (9)$$

where $\mathcal{D}_k$ is the k -th disturbance mode, π_k is its mixture weight, and θ_k is the corresponding parameters. The implemented wind modes include correlated stochastic wind, periodic wind, gust-dominated wind, step-like changes, Gaussian-like baseline perturbations, and heavy-tailed residual disturbances. During simulation, the selected wind vector is converted into a world-frame external force proportional to the vehicle mass and applied to the quadrotor base link. Wind therefore acts on translational motion, while attitude coupling is handled by the physics engine and the multirotor dynamics. Figure 2 compares the

marginal distribution of the x -axis wind velocity for the proposed multimodal disturbance and the Gaussian baseline disturbance. The multimodal wind shows multiple modes and relatively more pronounced tails, whereas the Gaussian baseline remains closer to a unimodal distribution.

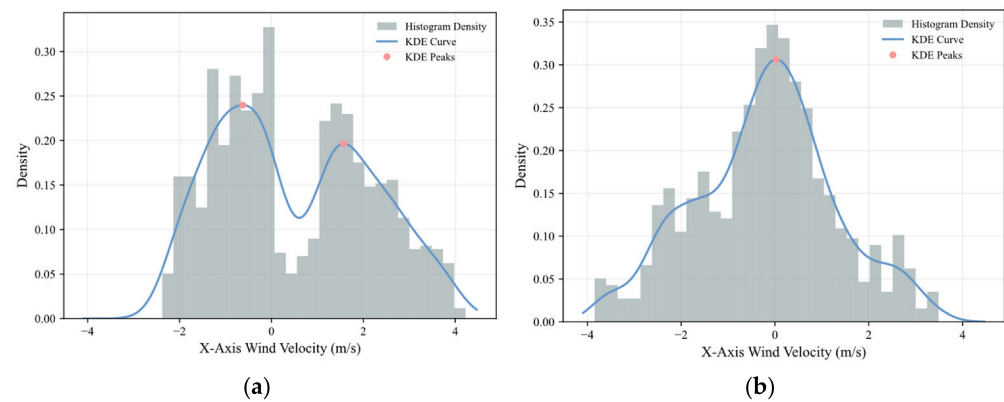


Figure 2. Comparison of the marginal distribution of the X -axis wind velocity under multimodal and Gaussian disturbance distributions. (a) Multimodal disturbance; (b) Gaussian disturbance.

3.3. Residual-Aware Deep Reinforcement Learning Policy

3.3.1. MDP Formulation

The trajectory-tracking problem is cast as a discounted Markov decision process (MDP). At step t the simulator state is $\mathbf{x}_t$, the policy observation is $\mathbf{o}_t$, the normalized control command is $\mathbf{u}_t$, and the scalar reward is r_t . The objective is to maximize the expected discounted return, as in (10):

$$\pi^* = \operatorname{argmax}_{\pi_\theta} \mathcal{J}(\theta), \quad \mathcal{J}(\theta) = \mathbb{E}_{\zeta \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \quad (10)$$

The baseline controller follows a PPO-style actor-critic structure, implemented with a standard parallel-rollback PPO setup. The actor and the critic are both multilayer perceptrons with layer normalization. Detailed optimizer and network hyperparameters are reported in Section 5.1.

Let $\rho_t(\theta)$ denote the probability ratio between the updated policy and the rollout policy. The probability ratio is given in (11) and the clipped PPO objective is given in (12):

$$\rho_t(\theta) = \pi_\theta(\mathbf{u}_t \mid \mathbf{o}_t) / \pi_{\theta_{old}}(\mathbf{u}_t \mid \mathbf{o}_t), \quad (11)$$

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t [\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)], \quad (12)$$

where A_t is the generalized advantage estimate and ϵ is the policy-update clipping parameter. The present work does not modify this optimizer. It modifies the disturbance distribution, the policy observation, and the reward terms so that the controller learns disturbance-aware tracking behavior under multimodal wind.

3.3.2. Online Acceleration-Residual Observation

The full simulator state covers the quadrotor pose, linear velocity, angular velocity, rotor-related quantities, the reference trajectory, and task statistics. The policy observation is a lower-dimensional vector built from relative future reference positions, vehicle velocity,

attitude-related axes, a finite action-history window, and the proposed disturbance residual feature. The base tracking observation is defined in (13):

$$\mathbf{o}_t^{\text{base}} = \left[\left(\Delta \mathbf{P}_t^{\text{ref}} \right)^\top, (\mathbf{v}_t)^\top, (\boldsymbol{\omega}_t)^\top, (\mathbf{b}_{x,t})^\top, (\mathbf{b}_{y,t})^\top, (\mathbf{b}_{z,t})^\top, (\mathbf{h}_t^u)^\top \right]^\top, \quad (13)$$

where $\Delta \mathbf{P}_t^{\text{ref}} = [\mathbf{p}_{t+1}^{\text{ref}} - \mathbf{p}_t, \dots, \mathbf{p}_{t+H_p}^{\text{ref}} - \mathbf{p}_t]$ stacks the H_p relative future reference positions, $\mathbf{p}_t$ is the quadrotor position, $\mathbf{v}_t$ the linear velocity, $\boldsymbol{\omega}_t$ the angular velocity, and $\mathbf{b}_{x,t}, \mathbf{b}_{y,t}, \mathbf{b}_{z,t}$ denote attitude-related body-axis vectors; $\mathbf{h}_t^u$ is the stacked action-history vector of length H_u . In the implementation, the policy observes $H_p = 10$ future trajectory points and $H_u = 5$ historical action steps.

Building on the base observation above, the proposed observation channel estimates an online acceleration residual from the vehicle response. It is used as a low-dimensional policy input feature rather than as a formal disturbance observer with convergence guarantees. Let $\mathbf{v}_t$ be the world-frame linear velocity, Δt the control period, $\mathbf{R}_t$ the attitude rotation matrix, $\mathbf{f}_t^B$ the body-frame collective thrust vector, m the nominal mass, $\mathbf{g}$ the gravity vector, and k_d a nominal quadratic-drag coefficient with unit kg m^{-1} . The measured and nominal accelerations are then approximated as in (14):

$$\mathbf{a}_t^{\text{meas}} = \frac{\mathbf{v}_t - \mathbf{v}_{t-1}}{\Delta t}, \quad \mathbf{a}_t^{\text{nom}} = \frac{1}{m} (\mathbf{R}_t \mathbf{f}_t^B) + \mathbf{g} - \frac{k_d}{m} \|\mathbf{v}_t\|_2 \mathbf{v}_t. \quad (14)$$

In this study, k_d is a fixed nominal-model parameter. When enabled, it is calibrated once by least-squares fitting $\mathbf{a}_{\text{meas}} - [m^{-1} \mathbf{R} \mathbf{f}^B + \mathbf{g}]$ against $\|\mathbf{v}\|_2 \mathbf{v}$ using wind-free flight data and is then shared by all policy variants. In the reported experiments, $k_d = 0 \text{ kg m}^{-1}$, so rotor and body drag remain in the lumped residual $\mathbf{d}_t$.

The instantaneous acceleration residual is then computed as in (15) by clipping the mismatch between the measured and nominal accelerations:

$$\mathbf{d}_t = \text{clip}(\mathbf{a}_t^{\text{meas}} - \mathbf{a}_t^{\text{nom}}, -c_d, c_d). \quad (15)$$

In (15), $\text{clip}(\cdot, -c_d, c_d)$ denotes the element-wise saturation of each of the three components of the residual vector to the interval $[-c_d, c_d]$, and $c_d > 0$ is a fixed clipping bound with the unit $\text{m} \cdot \text{s}^{-2}$; it is the same for all three axes, for all policy variants, and for training and evaluation. At the first step after reset, $\mathbf{d}_t$ is set to zero because no reliable previous velocity is available. The latest K residuals are stored in a sliding buffer and flattened into the disturbance feature as in (16):

$$\mathbf{z}_t^{\text{d}} = \text{vec}(\mathbf{d}_{t-K+1}, \dots, \mathbf{d}_t). \quad (16)$$

The final policy observation concatenates the base tracking observation and the residual feature, as in (17):

$$\mathbf{o}_t = \left[\left(\mathbf{o}_t^{\text{base}} \right)^\top, \left(\mathbf{z}_t^{\text{d}} \right)^\top \right]^\top. \quad (17)$$

The residual feature $\mathbf{z}_t^{\text{d}}$ carries wind-induced acceleration information, unmodeled aerodynamic effects, and other lumped model errors. Because $\mathbf{z}_t^{\text{d}}$ is low-dimensional, it can be appended to an existing multilayer-perceptron actor without introducing a recurrent estimator. To study the effect of disturbance-memory length, the residual observation window K is varied across the baseline (no residual) and several short-to-long residual histories.

The command $\mathbf{u}_t$ is the normalized collective-thrust and body-rate (CTBR) command accepted by the quadrotor model. In this work, the policy outputs a high-level continuous command consisting of a normalized collective-thrust channel and three normalized body-

rate channels, rather than direct motor-level signals. The command then passes through a low-level rate-control and control-allocation interface before it is applied to the multirotor dynamics; this interface, the rate controller, the mixer, and their relation to standard flight-control firmware are described in detail in Section 4.2. The learned policy therefore does not need to identify motor-level dynamics directly. Instead, it learns a compact rate-and-thrust tracking command that is executed by the low-level control interface. This choice improves training stability and keeps the action space comparable to common quadrotor RL benchmarks [26].

3.3.3. Residual-Input Regularization

Initial experiments showed that a rich residual history can speed up in-distribution learning, yet it may also push the policy to rely on wind-specific temporal patterns. This is consistent with the broader shortcut-learning risk reported in deep neural networks [28]. To control this risk, the residual input is regularized through three coordinated mechanisms.

First, a short residual window limits how much long-horizon wind-pattern information the actor sees, while still preserving recent mismatch information for feedback correction. In the main configuration, the window is set to $K = 1$ (one-step residual). Second, each residual component is clipped by c_d before it is stored in the buffer. Clipping prevents numerical transients and rare gust peaks from dominating the policy update. Third, during training, the entire residual feature is randomly masked with probability p_{drop} ; when masked, $\mathbf{z}_t^d$ is set to zero. Masking forces the policy to retain a useful baseline tracking behavior even when residual information is absent or unreliable. Residual dropout is disabled at deterministic evaluation time. Together, these mechanisms balance disturbance observability against the risk of overfitting to the training wind sequence.

3.3.4. Disturbance-Rejection Reward Design

The reward function uses four components: a position-tracking term, an action-smoothness term, an acceleration-smoothness term, and a CVaR-style tail-risk penalty. Position tracking is the dominant dense term. Action smoothness discourages abrupt command changes. Acceleration smoothness suppresses aggressive translational motion. The tail term penalizes excursions beyond an acceptable tracking-error threshold.

The Euclidean tracking error is defined in (5). The dense position-tracking reward is defined in (18):

$$r_t^{\text{pos}} = \eta^d \exp(-e_t). \quad (18)$$

The action-smoothness term regularizes the temporal variation in the normalized policy command and is defined in (19):

$$r_t^{\text{smooth}} = \eta_t^s \exp(-E_t^u) \cdot \mathbb{I}_{t>1}, \quad (19)$$

where E_t^u is the first-order action-change measure used by the implementation, η_t^s is a scheduled or clipped reward weight, and $\mathbb{I}_{t>1}$ avoids penalizing the first step after reset. The acceleration-smoothness term is defined in (20):

$$r_t^{\text{acc}} = \eta_t^{\text{acc}} \exp\left(-\|\mathbf{a}_t^{\text{lin}}\|_2\right), \quad (20)$$

where $\mathbf{a}_t^{\text{lin}}$ is the translational acceleration estimate used by the implementation, and η_t^{acc} is the corresponding weight.

Under multimodal wind disturbances, optimizing only the average tracking error can still allow rare but severe deviations. The reward therefore includes a CVaR-style thresholded quadratic tail penalty, defined in (21):

$$r_t^{\text{tail}} = \lambda_{\text{CVaR}} [\max(e_t - e_{\text{CVaR}}, 0)]^2, \quad (21)$$

where e_{CVaR} is a fixed tracking-error threshold that approximates an empirical upper-tail level of the in-distribution training error, and $\lambda_{\text{CVaR}} \geq 0$ is the CVaR-style tail-risk penalty weight. This formulation is consistent with the CVaR-based risk-sensitive control literature [21,22]. The threshold e_{CVaR} is calibrated once from the empirical error distribution of the baseline policy under the training wind family and is not selected on the evaluation sets. Specifically, the Euclidean tracking error is collected over in-distribution validation rollouts, its empirical 0.9 quantile is computed, and the threshold is rounded to a fixed value. In all reported experiments, $e_{\text{CVaR}} = 0.6$ m. The same threshold and λ_{CVaR} are shared by all variants. This calibration keeps the penalty inactive for most samples and active on the upper tail. If the threshold is too small, the tail term behaves like an additional dense tracking penalty and may encourage overly conservative control; if it is too large, the term is rarely activated and its tail-error benefit diminishes. The ablation in Section 5.4 examines this trade-off.

The per-step reward used for training combines the four components and is defined in (22):

$$r_t = r_t^{\text{pos}} + r_t^{\text{smooth}} + r_t^{\text{acc}} - r_t^{\text{tail}}. \quad (22)$$

The first three components reward tracking accuracy and motion smoothness, and the last component subtracts a penalty whenever the tracking error exceeds the threshold e_{CVaR} . When combined with the residual observation introduced in Section 3.3.2, the reward encourages the policy to use the residual feature not only for mean-error reduction but also for tail-error suppression. The ablation study in Section 5.4 verifies that the residual feature and the tail-risk penalty are complementary.

4. Implementation

4.1. Simulation Environment Setup

The implementation is built on the OmniDrones framework [29]. The original trajectory-tracking task is extended with a multimodal wind disturbance pool, residual observation buffers, residual dropout, a CVaR-style tail-risk penalty, and deterministic evaluation controls. The training environment runs many parallel environments to accelerate on-policy rollout collection. The task uses the Iris quadrotor model, a finite future-reference horizon, action-history features, and a relaxed tracking-error termination threshold during wind-disturbance training. Wind disturbance is enabled in multimodal mode and replayed from a pre-generated disturbance set.

The reference-trajectory generator supports several trajectory families. These include smooth lemniscate trajectories, chained polynomial trajectories, random zigzag trajectories, and the pointed figure-eight trajectory. During training, the task samples randomized polynomial and zigzag trajectories with vertical oscillation. This randomized design exposes the policy to diverse motion patterns, so that the controller learns trajectory-agnostic tracking behavior rather than overfitting to a fixed reference path. During evaluation, the task switches to a deterministic mode with a single environment, a fixed wind-pool index, a fixed phase offset, and a predefined reference trajectory. Residual dropout is disabled during evaluation so that policy differences are measured under identical disturbance and trajectory conditions.

4.2. Low-Level Control Interface and Control Allocation

The policy never commands the rotors directly. At every control step, the actor outputs a four-dimensional normalized command $\mathbf{u}_t = [u_t^T, (\mathbf{u}_t^\omega)^\top]^\top \in [-1, 1]^4$, in which u_t^T is the normalized collective-thrust channel and $\mathbf{u}_t^\omega \in [-1, 1]^3$ collects the normalized roll-rate, pitch-rate, and yaw-rate channels. This command is denormalized into a physical CTBR setpoint, as in (23):

$$T_{des} = \frac{1}{2}(1 + u_t^T)T_{max}, \quad \boldsymbol{\omega}_{des} = \boldsymbol{\omega}_{max} \odot \mathbf{u}_t^\omega, \quad (23)$$

where T_{des} is the desired collective thrust, T_{max} is the maximum collective thrust of the vehicle, $\boldsymbol{\omega}_{des}$ is the desired body-rate vector, $\boldsymbol{\omega}_{max}$ collects the per-axis maximum body rates, and $\odot$ denotes the element-wise (Hadamard) product. The CTBR setpoint is held constant over one policy control period and is tracked by a low-level body-rate controller that runs at the physics rate of the simulator, which is faster than the policy rate. The rate controller is a proportional-integral law with the gyroscopic feed-forward term of the rigid-body model in (4), as given in (24):

$$\boldsymbol{\tau}_{des} = \mathbf{J} \left(\mathbf{K}_P \mathbf{e}_\omega + \mathbf{K}_I \int \mathbf{e}_\omega dt \right) + \boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega}, \quad \mathbf{e}_\omega = \boldsymbol{\omega}_{des} - \boldsymbol{\omega}, \quad (24)$$

where $\mathbf{e}_\omega$ is the body-rate error, $\mathbf{K}_P$ and $\mathbf{K}_I$ are diagonal rate-loop gain matrices, and $\boldsymbol{\tau}_{des}$ is the desired body torque. The desired collective thrust and the desired body torque are then converted into individual rotor forces by a static control-allocation (mixing) step. For the X-configuration quadrotor model used in this work, with arm length l and rotor torque-to-thrust ratio c_τ , the wrench generated by the four rotor forces $\mathbf{f} = (f_1, f_2, f_3, f_4)^\top$ is a linear function of $\mathbf{f}$, and the allocation is obtained by inverting this relation, as in (25):

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \mathbf{M} \mathbf{f}, \quad \mathbf{M} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -l & -l & l & l \\ -l & l & l & -l \\ -c_\tau & c_\tau & -c_\tau & c_\tau \end{bmatrix}, \quad \mathbf{f} = \mathbf{M}^{-1} \begin{bmatrix} T_{des} \\ \boldsymbol{\tau}_{des} \end{bmatrix}, \quad (25)$$

where T and (τ_x, τ_y, τ_z) are the collective thrust and the body torques actually generated by the rotors, and the signs of the entries of the mixing matrix $\mathbf{M}$ follow the rotor ordering and the spin directions of the vehicle model. Each allocated rotor force f_i is saturated to the admissible interval $[0, f_{max}]$ before it is applied, so that the commanded wrench is always physically realizable; the saturated rotor forces are then passed to the physics engine, which integrates the dynamics in (1)–(4).

This interface is deliberately aligned with the rate-control path of standard flight-control firmware. In PX4, for instance, the position and attitude loops produce exactly such a CTBR setpoint, which is consumed by the multicopter rate-control module and then by the control allocator, which maps the thrust and torque setpoints into individual motor commands [30]. The proposed policy replaces the position and attitude loops of the firmware, whereas the rate loop and the mixer are left unchanged; deploying the policy on such a stack therefore requires no modification of the low-level firmware and no motor-level system identification, and the residual feature in (14) and (15) can be computed from signals that are already available on board. The same CTBR interface is used by the SimpleFlight baseline [26], so all compared variants share an identical action space and the comparison in Section 5 is independent of motor-level modeling details.

4.3. Wind Disturbance Pool Construction

The wind disturbance pool is generated offline before training or validation. Offline generation avoids repeated startup costs and ensures that different runs share identical disturbance realizations. Each generated set stores the wind tensor $\mathbf{B} \in \mathbb{R}^{N \times L \times 3}$, where N is the number of wind sequences, L is the sequence length, and the last dimension carries the three Cartesian wind components. $\mathcal{W}_{\text{train}}$ denotes the finite wind pool sampled offline from $\mathcal{P}_w^{\text{train}}$, while $\mathbf{B}$ denotes the tensor that stores those samples. At each environment reset, one wind sequence is drawn uniformly from $\mathcal{W}_{\text{train}}$. Table 1 summarizes the wind set usage.

Table 1. Wind disturbance pool split used for training and validation.

Source	Use	Description
Training wind set	Policy learning	Large multimodal wind set generated from the training disturbance family.
ID validation set	In-distribution checkpoint selection and validation	Wind sequences from the training family with pronounced directional variation.
OOD validation set	Out-of-distribution robustness testing	Wind set generated with adjusted parameters that fall outside the training distribution.

The training set contains a large multimodal wind family used for policy learning. The in-distribution (ID) evaluation set is selected from the same wind-generation family, with pronounced directional variation, which enables deterministic checkpoint comparison under distributionally consistent wind. The out-of-distribution (OOD) evaluation set is generated with adjusted disturbance-generation parameters: several amplitude and temporal-variation settings are moderately increased relative to the training configuration, which places the set outside the training distribution and makes it slightly more difficult.

4.4. Training Procedure

Algorithm 1 summarizes the rollout and update procedure used in the implementation. At each environment reset, a reference trajectory and a wind sequence are sampled from the trajectory generator and the wind disturbance pool, respectively, and the simulator state and the residual buffer are reinitialized. At each control step, the base tracking observation in (13) is combined with the residual feature in (16)–(17) and is fed to the actor; the residual feature is randomly masked with probability p_{drop} . After the simulator advances under the realized wind force, the per-step reward in (22) is evaluated. Collected rollouts then update the actor and the critic using the PPO objective in (12). The procedure is repeated for a fixed number of training iterations.

The policies are trained using the PPO-style update described in Section 3.3.1 on the OmniDrones parallel-environment infrastructure, following the rollout flow in Algorithm 1. Each training run uses many parallel environments, a fixed maximum episode length, and a fixed total number of training iterations. Each environment samples a randomized reference trajectory together with a wind sequence from the training wind set at every reset. The simulator then advances under the corresponding wind force, and the rollouts update the actor and the critic with the PPO clipped objective in (12). Detailed training hyperparameters are reported in Section 5.1.

Algorithm 1: Training procedure of the proposed residual-aware DRL framework.

```

Require: wind pool  $\mathcal{W}_{\text{train}}$  (sampled from  $\mathcal{P}_w^{\text{train}}$ ); reference generator  $\mathcal{G}_{\text{ref}}$ ; initial
1 policy  $\pi_\theta$  and value  $V_\phi$ ; residual window  $K$ ; clip bound  $c_d$ ; dropout probability
 $p_{\text{drop}}$ ; iterations  $N_{\text{iter}}$ ; parallel environments  $N_{\text{env}}$ ; horizon  $T$ .
2 initialize parameters  $\theta$  and  $\phi$ 
3 for  $i = 1$  to  $N_{\text{iter}}$  do
4   initialize empty rollout buffer  $\mathcal{D}$ 
5   for each parallel environment  $n = 1, \dots, N_{\text{env}}$  do
6     sample  $\mathbf{p}_{0:T-1}^{\text{ref}} \sim \mathcal{G}_{\text{ref}}$  and  $\mathbf{w}_{0:T-1} \sim \mathcal{W}_{\text{train}}$ 
7     reset state  $\mathbf{x}_0$ ; clear residual window ( $\mathbf{d} \leftarrow \mathbf{0}$ )
8     for  $t = 0$  to  $T - 1$  do
9        $\mathbf{o}_t^{\text{base}} = f_{\text{track}}(x_t, \mathbf{p}_{t+1:t+H_p}^{\text{ref}}, \mathbf{u}_{t-H_u:t-1})$ 
10       $\mathbf{d}_t = \text{clip}(\mathbf{a}_t^{\text{meas}} - \mathbf{a}_t^{\text{nom}}, -c_d, c_d)$ 
11       $\mathbf{z}_t^{\text{d}} = \text{vec}(\mathbf{d}_{t-K+1}, \dots, \mathbf{d}_t)$ ; with probability  $p_{\text{drop}}$  set  $\mathbf{z}_t^{\text{d}} \leftarrow \mathbf{0}$ 
12       $\mathbf{o}_t = [(\mathbf{o}_t^{\text{base}})^\top, (\mathbf{z}_t^{\text{d}})^\top]^\top$ ; sample  $\mathbf{u}_t \sim \pi_\theta(\cdot | \mathbf{o}_t)$ 
13      execute  $\mathbf{u}_t$  via the CTBR interface; advance simulator under  $\mathbf{w}_t$ 
14      observe next state; compute reward  $r_t$ 
15      store  $(\mathbf{o}_t, \mathbf{u}_t, r_t, \log \pi_\theta(\mathbf{u}_t | \mathbf{o}_t))$  in  $\mathcal{D}$ 
16    end for
17  end for
18  compute GAE advantages  $\{A_t\}$  and returns from  $\mathcal{D}$ 
19  for epoch = 1 to  $N_{\text{epoch}}$  do
20    update  $\theta$  by maximizing  $\mathcal{L}^{\text{CLIP}}(\theta)$ ; update  $\phi$  by the value loss
21  end for
22  if  $i \bmod C = 0$  then save checkpoint for ID validation
23 end for
24 Ensure: policy  $\pi_{\theta^*}$  selected by the validation protocol.

```

Checkpoint selection is decoupled from final evaluation. Training checkpoints are saved every fixed number of iterations and evaluated on the ID validation set. Each checkpoint is run deterministically on the smooth lemniscate trajectory under a representative multimodal wind sequence, and its average tracking error is recorded. The checkpoint with the lowest validation error is selected as the deployed policy for that variant. The OOD wind set and the measured real-wind set in Section 5 are never used for checkpoint selection. This protocol avoids selecting models on the OOD test set and is important for disturbance-rejection reinforcement learning. Rich residual histories can lower the training error quickly, but they may overfit to wind-specific temporal patterns. A separate validation set is therefore needed to select checkpoints that retain tracking accuracy without relying on such shortcuts.

5. Experimental Results and Discussion

This section evaluates the proposed method on the OmniDrones-based quadrotor trajectory-tracking task. The experiments use two validation trajectories: the smooth lemniscate and the pointed figure-eight. Both ID and OOD wind disturbances are tested, an ablation separates the effects of the residual feature and the tail-risk penalty, and a real wind test based on the Gulf of Eilat dataset [31] checks practical robustness.

5.1. Experimental Setup

SimpleFlight is the baseline controller [26]. It is built on OmniDrones and provides a standard DRL pipeline for quadrotor trajectory tracking. The proposed residual-aware policy keeps the same PPO-style control backbone and modifies only the observation, reward, disturbance distribution, and residual regularization.

To quantify the effect of the residual-window length and select the window used in the proposed method, four configurations of the same residual-aware framework are evaluated: RA-W1, RA-W2, RA-W3, and RA-W8. Here, RA-WK denotes a residual-aware policy with a residual observation window of K steps, and RA-W1 is the compact configuration adopted as the main method. These four configurations are treated as a parameter-sensitivity study. SimpleFlight is retained separately as the external baseline and uses the standard tracking observation without residual input.

All RA-WK variants use the same PPO backbone, actor and critic hidden-layer depth and width, layer normalization, optimizer and PPO hyperparameters, number of parallel environments, episode length, training iterations, reference-trajectory generator, training wind pool, reward function and weights, residual clipping bound c_d , residual-dropout probability p_{drop} , and checkpoint-selection rule on the ID validation set. Changing K necessarily increases the observation dimension by $3K$ and changes only the first linear layer of the actor and critic; all subsequent layers are unchanged. Because the observation dimension grows with K , the parameter count increases with K . The difference is confined to the first linear layers of the actor and critic, while all subsequent layers remain unchanged. Thus, the comparison preserves the same network architecture across variants, with only a small parameter-count difference induced by the residual-window dimension. Their training convergence and held-out robustness are reported separately in Sections 5.2 and 5.3.

All experiments were conducted on a workstation running Ubuntu 20.04 LTS. The hardware comprises an Intel Core i9-14900K CPU with 32 threads, 128 GB of system RAM, and an NVIDIA RTX 5880 Ada Generation GPU with 48 GB of VRAM. The workstation uses NVIDIA driver 575.57.08 and supports CUDA 12.9. The implementation is based on Python 3.7.16 and PyTorch 1.13.0. Each training run uses 8192 parallel environments, a maximum episode length of 1000 control steps, and 12,000 total training iterations. During training, the policy collects on-policy rollouts under the multimodal wind disturbance pool and updates the actor and the critic with the PPO-style implementation described in Section 3.3.1.

Two reference trajectory types are used to examine whether the trained policy can follow paths with different geometric shapes and dynamic difficulty. Figure 3 shows the two trajectories. The smooth three-dimensional lemniscate acts as a nominal closed-curve validation task. The pointed figure-eight keeps a figure-eight topology but approximates the curve through straight segments, which introduces sharp corner transitions and serves as a sharp-corner stress test. Table 2 summarizes the two trajectory configurations and their roles. Here, T_{cyc} denotes the period of one reference-trajectory cycle.

The main quantitative metrics include four indicators: average tracking error (Avg. Error), 95th percentile tracking error (E_{95} Error), upper-tail mean error at the 0.9 quantile (CVaR_{0.9} Error), and progress completion ratio (PCR). Let e_t denote the Euclidean tracking error defined in (5), and let $\mathcal{I}$ denote the set of valid evaluation time indices across all evaluated trajectories. Equations (26)–(29) define these four metrics:

$$E_{\text{avg}} = \frac{1}{N} \sum_{t \in \mathcal{I}} e_t; \quad (26)$$

$$E_{95} = Q_{0.95}\{e_t : t \in \mathcal{I}\}; \quad (27)$$

$$E_{\text{CVaR}0.9} = \frac{1}{|\mathcal{T}_{0.9}|} \sum_{t \in \mathcal{T}_{0.9}} e_t, \quad \mathcal{T}_{0.9} = \{t \in \mathcal{I} : e_t > Q_{0.9}\{e_t : t \in \mathcal{I}\}\}; \quad (28)$$

$$R_{\text{PCR}} = \frac{\sum_{i=1}^N n_i^{\text{done}}}{\sum_{i=1}^N n_i^{\text{ref}}} \times 100\%. \quad (29)$$

In (29), n_i^{done} is the number of completed progress steps of the i -th evaluation trajectory, and n_i^{ref} is the corresponding planned total. The progress completion ratio is continuous at the trajectory-aggregate level and serves as the task-completion metric for the evaluation set.

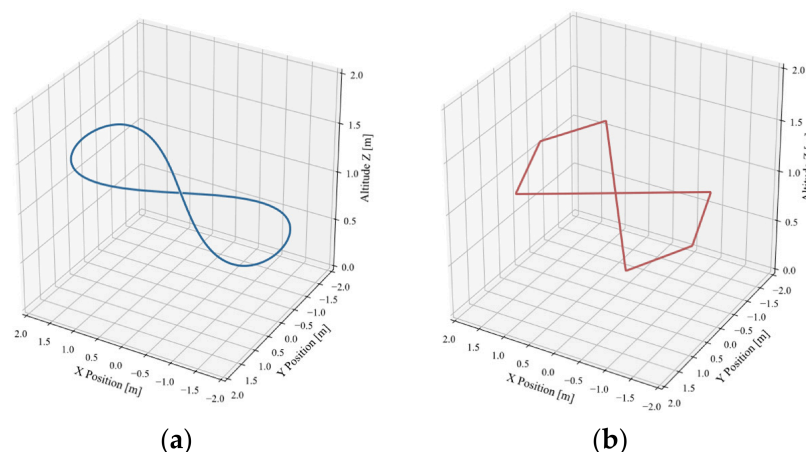


Figure 3. Planned reference trajectories for trajectory-level evaluation. (a) Smooth three-dimensional lemniscate used as the nominal closed-curve tracking task; (b) pointed figure-eight reference composed of straight segments and used as the sharp-corner stress-test task.

Table 2. Evaluation trajectories used for trajectory-level validation.

Trajectory	Current Configuration	Role and Purpose
Smooth lemniscate	$T_{\text{cyc}} = 7.0$ s, maximum absolute x/y displacement = 2.0 m, z amplitude = 0.3 m.	Nominal smooth closed-curve validation for steady tracking.
Pointed figure-eight	$T_{\text{cyc}} = 7.0$ s, maximum absolute x/y displacement = 1.5 m, z amplitude = 0.3 m, eight straight segments.	Sharp-corner stress test for transient tracking under abrupt reference-direction changes.

5.2. Training Results

Figure 4 reports the training convergence of the tracking error for the five policy variants under the multimodal training wind set. Residual information can accelerate training, but the effect depends strongly on how much residual history is exposed to the policy. Among the residual-window settings, RA-W8 decreases the fastest and reaches the lowest stabilized training error, while RA-W1 and RA-W2 maintain slower but more stable training curves. The SimpleFlight policy, which receives no residual feature, shows the slowest convergence and the largest steady-state training error.

Training-time convergence alone does not determine deployment robustness. To examine this, checkpoints are saved every 100 training iterations and evaluated under the validation protocol of Section 4.4. Each saved checkpoint is tested on the smooth lemniscate under a representative multimodal wind sequence, and the resulting tracking errors are reported in Figure 5.

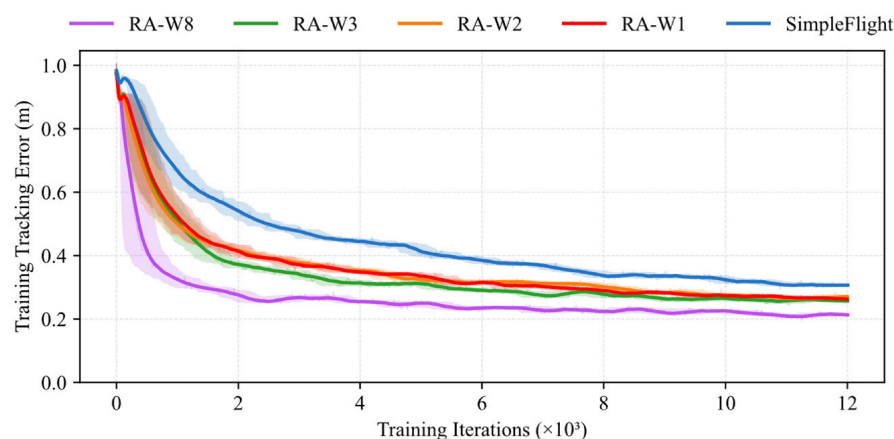


Figure 4. Training convergence of tracking error for different policy variants under multimodal wind disturbances.

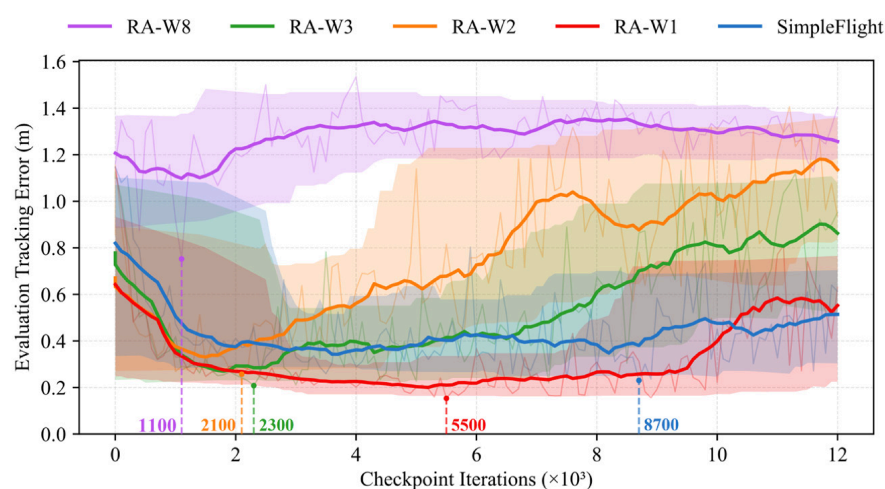


Figure 5. Checkpoint evaluation of smooth-lemniscate tracking error versus training iterations under the representative multimodal wind sequence.

Figure 5 reveals a clear mismatch between training convergence and checkpoint robustness. The best validation checkpoints occur at 8700 iterations for SimpleFlight, 5500 for RA-W1, 2100 for RA-W2, 2300 for RA-W3, and 1100 for RA-W8. Although RA-W8 learns the fastest and reaches the smallest training error, its checkpoint-level robustness peaks early and degrades afterwards. This pattern is consistent with the shortcut-learning concern raised in Section 3.3.3: a long residual history can allow the policy to exploit wind-specific temporal patterns in the training set, which raises the training reward but harms held-out robustness. The compact RA-W1 variant, by contrast, maintains a monotonically improving validation curve over a wider iteration range.

5.3. Validation Results

Tables 3 and 4 report the residual observation window study under the ID and OOD wind sets, respectively. The comparison focuses on two questions: whether residual observation improves average tracking accuracy, and whether increasing the residual-memory length improves robustness. Error metrics are reported as mean (standard deviation) across the evaluation wind sequences, and PCR is the aggregate progress completion ratio.

Table 3. In-distribution (ID) disturbance-residual observation window study.

Trajectory	Variant	Avg. Error (m)	E_{95} Error (m)	CVaR _{0.9} Error (m)	PCR
Smooth lemniscate	SimpleFlight	0.369 (0.142)	1.019 (0.491)	1.050 (0.474)	84.25%
	RA-W1	0.186 (0.039)	0.595 (0.222)	0.666 (0.192)	100.00%
	RA-W2	0.350 (0.137)	1.000 (0.410)	1.057 (0.379)	94.23%
	RA-W3	0.282 (0.107)	0.856 (0.365)	0.950 (0.337)	95.04%
	RA-W8	0.875 (0.119)	1.908 (0.213)	1.950 (0.162)	8.73%
Pointed figure-eight	SimpleFlight	0.436 (0.143)	1.094 (0.446)	1.141 (0.443)	73.49%
	RA-W1	0.226 (0.054)	0.605 (0.203)	0.649 (0.185)	98.54%
	RA-W2	0.434 (0.135)	1.141 (0.396)	1.226 (0.355)	76.01%
	RA-W3	0.307 (0.072)	0.880 (0.294)	0.928 (0.277)	96.73%
	RA-W8	0.811 (0.120)	1.877 (0.260)	1.913 (0.213)	9.97%

Note: “Avg. Error” is the average Euclidean tracking error E_{avg} defined in (26); “ E_{95} Error” is the 95th percentile tracking error defined in (27); “CVaR_{0.9} Error” is the conditional value-at-risk of the tracking error at level 0.9, that is, the mean of the errors that exceed their own 0.9 quantile, defined in (28); and “PCR” is the progress completion ratio defined in (29). The three error metrics are reported in meters as mean (standard deviation) across the evaluation wind sequences, PCR is an aggregate percentage, and bold entries mark the best value of a column for a given trajectory. Lower is better for the three error metrics and higher is better for PCR.

Table 4. Out-of-distribution (OOD) disturbance-residual observation window study.

Trajectory	Variant	Avg. Error (m)	E_{95} Error (m)	CVaR _{0.9} Error (m)	PCR
Smooth lemniscate	SimpleFlight	0.389 (0.124)	1.013 (0.439)	1.056 (0.417)	82.68%
	RA-W1	0.252 (0.100)	0.779 (0.453)	0.819 (0.429)	94.39%
	RA-W2	0.397 (0.159)	1.073 (0.384)	1.129 (0.354)	89.61%
	RA-W3	0.342 (0.132)	1.048 (0.441)	1.096 (0.435)	92.04%
	RA-W8	0.854 (0.153)	1.898 (0.201)	1.933 (0.181)	7.73%
Pointed figure-eight	SimpleFlight	0.394 (0.142)	0.908 (0.403)	0.958 (0.410)	88.66%
	RA-W1	0.236 (0.070)	0.694 (0.362)	0.716 (0.356)	100.00%
	RA-W2	0.357 (0.124)	0.925 (0.347)	0.984 (0.332)	94.71%
	RA-W3	0.253 (0.054)	0.682 (0.246)	0.712 (0.239)	99.71%
	RA-W8	0.797 (0.130)	1.910 (0.226)	1.944 (0.190)	10.16%

Note: Bold entries mark the best value of a column for a given trajectory. Lower is better for the three error metrics and higher is better for PCR.

The ID results in Table 3 show that RA-W1 gives the best overall performance on both trajectories. On the smooth lemniscate, RA-W1 reduces Avg. Error from 0.369 to 0.186 (49.6% reduction), E_{95} Error from 1.019 to 0.595 (41.6% reduction), and CVaR_{0.9} Error from 1.050 to 0.666 (36.6% reduction), while raising PCR from 84.25% to 100.00%. On the pointed figure-eight, RA-W1 reduces Avg. Error from 0.436 to 0.226 (48.2%), E_{95} Error from 1.094 to 0.605 (44.7%), and CVaR_{0.9} Error from 1.141 to 0.649 (43.1%), and raises PCR from 73.49% to 98.54%, a gain of 25.05 percentage points. A compact one-step residual therefore improves both the mean error and the tail error under the ID wind family.

The OOD results in Table 4 follow the same overall trend, although the margins are smaller and the trajectory geometry plays a clearer role. RA-W1 remains the most consistent variant. On the smooth lemniscate, it reduces Avg. Error from 0.389 to 0.252 (35.2%), E_{95} Error from 1.013 to 0.779 (23.1%), and CVaR_{0.9} Error from 1.056 to 0.819 (22.4%), while raising PCR from 82.68% to 94.39%. On the pointed figure-eight, it reduces Avg. Error from 0.394 to 0.236 (40.1%) and raises PCR from 88.66% to 100.00%. Although RA-W3 gives a slightly lower E_{95} Error and CVaR_{0.9} Error on this single OOD pointed case, the differences from RA-W1 are small relative to the reported across-sequence variability and do not extend to mean error or progress completion. RA-W3 is also clearly less stable on the smooth lemniscate and under ID wind. Therefore, this local near-tie does not contradict the main observation that RA-W1 offers the most balanced checkpoint-level robustness across the four evaluation cases.

The long-history RA-W8 variant clearly contradicts the assumption that greater residual memory always improves performance. Although its training curve decreases fastest, Tables 3 and 4 show very large evaluation errors and very low PCR values for RA-W8 across both trajectories and wind sets. The E_{95} and $CVaR_{0.9}$ values remain around 1.9 in all four cases.

The experimental result is that, among the tested configurations, longer residual windows do not improve held-out robustness, even though RA-W8 reduces the training error rapidly. A longer residual history exposes the policy to more temporal information, and therefore provides more opportunities to fit correlations that are specific to the training wind realizations; such correlations can improve the training objective without transferring to held-out wind. The early validation peak and subsequent deterioration of RA-W8, together with the benefit of a short window and residual dropout, are consistent with this shortcut-learning interpretation [28]. However, shortcut learning is not directly measured in the present study, and the larger input dimension or accumulated residual noise may also explain part of the degradation. Therefore, the result supports only the limited conclusion that $K = 1$ gives the best balance under the tested settings. Direct representation analysis and counterfactual wind experiments are left for future work.

Figure 6 presents representative trajectory-tracking examples under the same fixed held-out OOD wind sequence for both the smooth lemniscate and the pointed figure-eight references. In the horizontal plane, +x points east and +y points north. Consistent with the overall trends in Table 4, RA-W1 generally follows the reference more closely than SimpleFlight and avoids the severe degradation observed in RA-W8. The lower bar plot shows the wind-speed variation over time, with each time segment averaged every five simulation steps; the color bar encodes the horizontal wind direction.

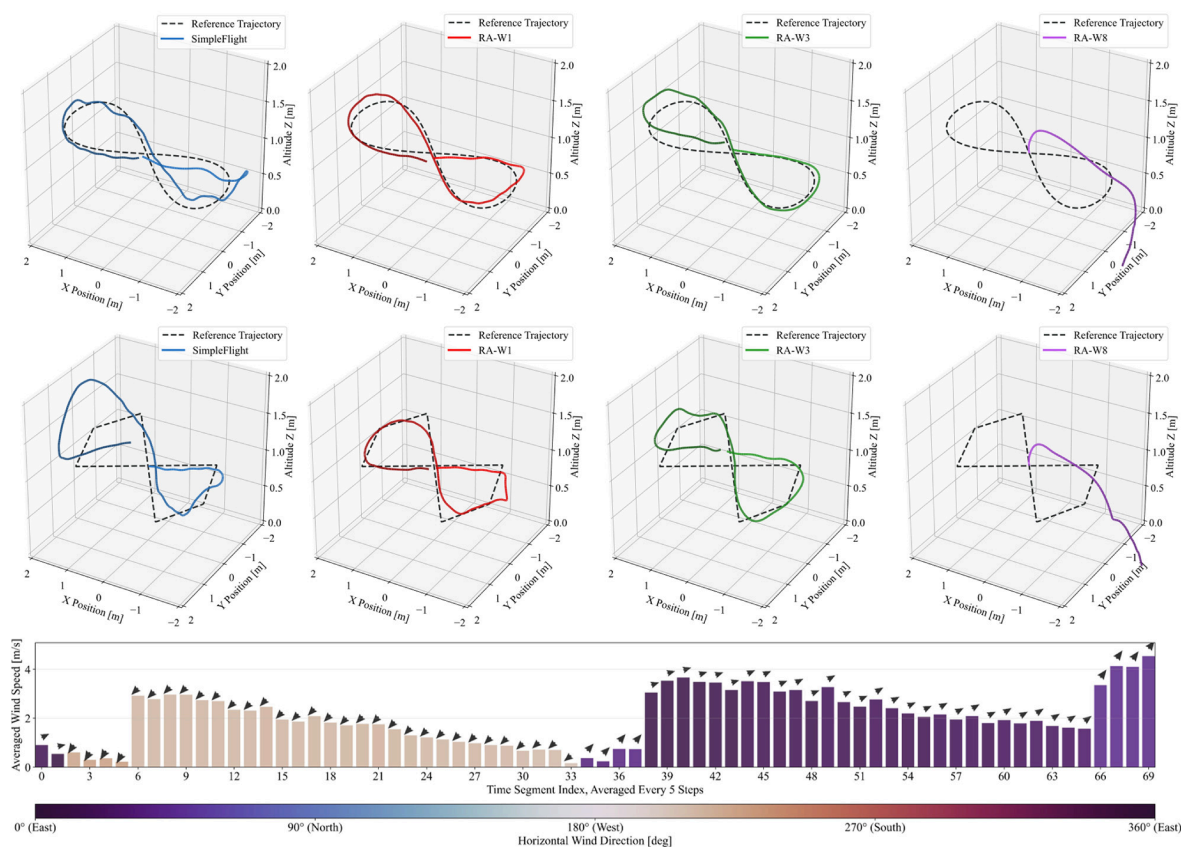


Figure 6. Representative qualitative trajectory-tracking examples under the same fixed held-out OOD wind sequence for the smooth lemniscate and pointed figure-eight references. The lower bar plot shows the averaged wind-speed sequence, and the color bar represents the horizontal wind direction.

5.4. Ablation Study

The ablation study uses the same checkpoint selection protocol as Section 4.4. Four variants are compared in Tables 5 and 6: the SimpleFlight policy, a residual-observation-only policy (denoted + dist_obs), a tail-penalty-only policy (denoted + tail penalty), and the full compact residual-aware method RA-W1, which combines both components. After training, checkpoints are selected on the ID validation set and then evaluated on both the ID and OOD wind sets.

Table 5. In-distribution (ID) ablation study of residual observation and CVaR-style tail-risk penalty.

Trajectory	Variant	Avg. Error (m)	E_{95} Error (m)	CVaR _{0.9} Error (m)	PCR
Smooth lemniscate	SimpleFlight	0.369 (0.142)	1.019 (0.491)	1.050 (0.474)	84.25%
	+ dist_obs	0.373 (0.181)	1.132 (0.513)	1.197 (0.494)	85.03%
	+ tail penalty	0.241 (0.069)	0.751 (0.353)	0.804 (0.313)	98.91%
	RA-W1	0.186 (0.039)	0.595 (0.222)	0.666 (0.192)	100.00%
Pointed figure-eight	SimpleFlight	0.436 (0.143)	1.094 (0.446)	1.141 (0.443)	73.49%
	+ dist_obs	0.329 (0.133)	0.898 (0.427)	0.962 (0.405)	79.96%
	+ tail penalty	0.324 (0.103)	0.970 (0.374)	1.007 (0.352)	94.44%
	RA-W1	0.226 (0.054)	0.605 (0.203)	0.649 (0.185)	98.54%

Note: Bold entries mark the best value of a column for a given trajectory. Lower is better for the three error metrics and higher is better for PCR.

Table 6. Out-of-distribution (OOD) ablation study of residual observation and CVaR-style tail-risk penalty.

Trajectory	Variant	Avg. Error (m)	E_{95} Error (m)	CVaR _{0.9} Error (m)	PCR
Smooth lemniscate	SimpleFlight	0.389 (0.124)	1.013 (0.439)	1.056 (0.417)	82.68%
	+ dist_obs	0.352 (0.158)	1.082 (0.434)	1.132 (0.411)	88.17%
	+ tail penalty	0.272 (0.080)	0.767 (0.338)	0.835 (0.315)	96.30%
	RA-W1	0.252 (0.100)	0.779 (0.453)	0.819 (0.429)	94.39%
Pointed figure-eight	SimpleFlight	0.394 (0.142)	0.908 (0.403)	0.958 (0.410)	88.66%
	+ dist_obs	0.303 (0.189)	0.829 (0.445)	0.859 (0.431)	97.92%
	+ tail penalty	0.317 (0.111)	0.890 (0.379)	0.932 (0.381)	96.05%
	RA-W1	0.236 (0.070)	0.694 (0.362)	0.716 (0.356)	100.00%

Note: Bold entries mark the best value of a column for a given trajectory. Lower is better for the three error metrics and higher is better for PCR.

This design separates the effect of disturbance awareness from the effect of tail-risk optimization. The residual-observation-only variant tests whether a one-step residual feature is useful on its own. The tail-penalty-only variant tests whether suppressing rare high-error excursions is sufficient without any residual input. RA-W1 then tests whether these two mechanisms are complementary across the smooth lemniscate and the pointed figure-eight trajectories.

In the ID ablation in Table 5, the full RA-W1 configuration gives the lowest error metrics and the strongest PCR across both trajectories. On the smooth lemniscate, the tail-penalty-only variant reduces Avg. Error from 0.369 to 0.241 (34.7%); RA-W1 pushes it further to 0.186 (49.6%) and reaches 100.00% PCR. On the pointed figure-eight, residual observation alone (+ dist_obs) and tail-penalty alone (+ tail penalty) both reduce the mean error relative to SimpleFlight, but neither matches the full RA-W1 across the three error metrics and PCR. RA-W1 reduces Avg. Error to 0.226, E_{95} Error to 0.605, and CVaR_{0.9} Error to 0.649 (reductions of 48.2%, 44.7%, and 43.1%, respectively), and raises PCR to 98.54%.

The OOD ablation in Table 6 confirms that the residual feature and the CVaR-style tail-risk penalty are complementary. On the smooth lemniscate, residual observation alone reduces Avg. Error from 0.389 to 0.352 (9.5%) and raises PCR from 82.68% to 88.17%,

but its tail errors remain higher than the baseline. The tail-penalty-only variant is more effective on the smooth lemniscate, reducing Avg. Error by 30.1% and CVaR_{0.9} Error by 20.9%; however, RA-W1 gives the lowest smooth-trajectory Avg. Error and the lowest CVaR_{0.9} Error among the compared variants. On the pointed figure-eight, RA-W1 reduces Avg. Error by 40.1% while maintaining 100.00% PCR. The proposed method is not simply the result of adding residual information; it depends on combining compact residual observation with CVaR-style tail-risk penalty and with the checkpoint-selection protocol.

5.5. Real Wind Disturbance Tests

To examine whether the proposed residual-aware policy remains useful beyond synthetic wind, a real wind disturbance test is introduced using the public Gulf of Eilat wind-waves interaction dataset collected at the Interuniversity Institute for Marine Sciences [31]. The measured wind time series is resampled to the simulation control period, converted into three-axis acceleration disturbance sequences after removing missing or invalid samples, and normalized to match the force-injection interface used in the OmniDrones environment. Compared with the generated synthetic wind, the measured wind exhibits stronger temporal correlation, while its marginal distribution shows both Gaussian-like and multimodal characteristics.

As shown by a representative 7.0 s segment in Figure 7, the wind components along the x and y axes do not change abruptly during the segment, yet the wind-speed distribution is diverse: the x -axis component is closer to a Gaussian-like distribution, whereas the y -axis component shows a more multimodal pattern. For this test, 20 measured wind trajectories are selected, and three consecutive 7.0 s segments are extracted from each trajectory, which yields 60 measured wind disturbance sequences. The same quadrotor model, checkpoint-selection rule, evaluation trajectories, and four metrics used in Sections 5.3 and 5.4 are retained, so that the real wind test is directly comparable with the ID and OOD evaluations.

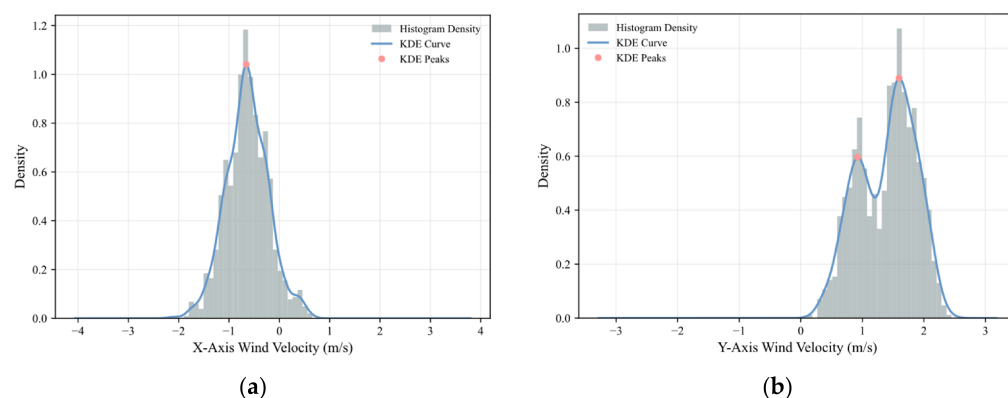


Figure 7. Marginal distribution of a representative measured real-wind component from a 7.0 s Gulf of Eilat segment. (a) X-axis wind component. (b) Y-axis wind component.

During this test, the quadrotor tracks the smooth lemniscate and pointed figure-eight reference trajectories under replayed measured wind segments. Each segment is treated as a fixed disturbance realization, and the policy is evaluated deterministically without residual dropout. The setting probes whether the compact residual feature can react to realistic temporal wind fluctuations without relying on the synthetic wind-mode labels used during training. Quantitative results are reported in Table 7, and Figure 8 provides representative tracking cases together with the corresponding measured-wind profile. In the figure, the lower panel gives the five-step averaged wind speed, and the color bar indicates the horizontal wind direction. Error metrics are reported as mean (standard deviation), and PCR is again the aggregate progress completion ratio.

Table 7. Real wind disturbance test based on the Gulf of Eilat wind-waves interaction dataset.

Trajectory	Variant	Avg. Error (m)	E_{95} Error (m)	CVaR _{0.9} Error (m)	PCR
Smooth lemniscate	SimpleFlight	0.324 (0.096)	0.807 (0.375)	0.855 (0.368)	97.41%
	RA-W1	0.224 (0.034)	0.587 (0.176)	0.684 (0.114)	100.00%
Pointed figure-eight	SimpleFlight	0.301 (0.075)	0.687 (0.246)	0.735 (0.294)	83.78%
	RA-W1	0.245 (0.056)	0.585 (0.232)	0.626 (0.242)	99.25%

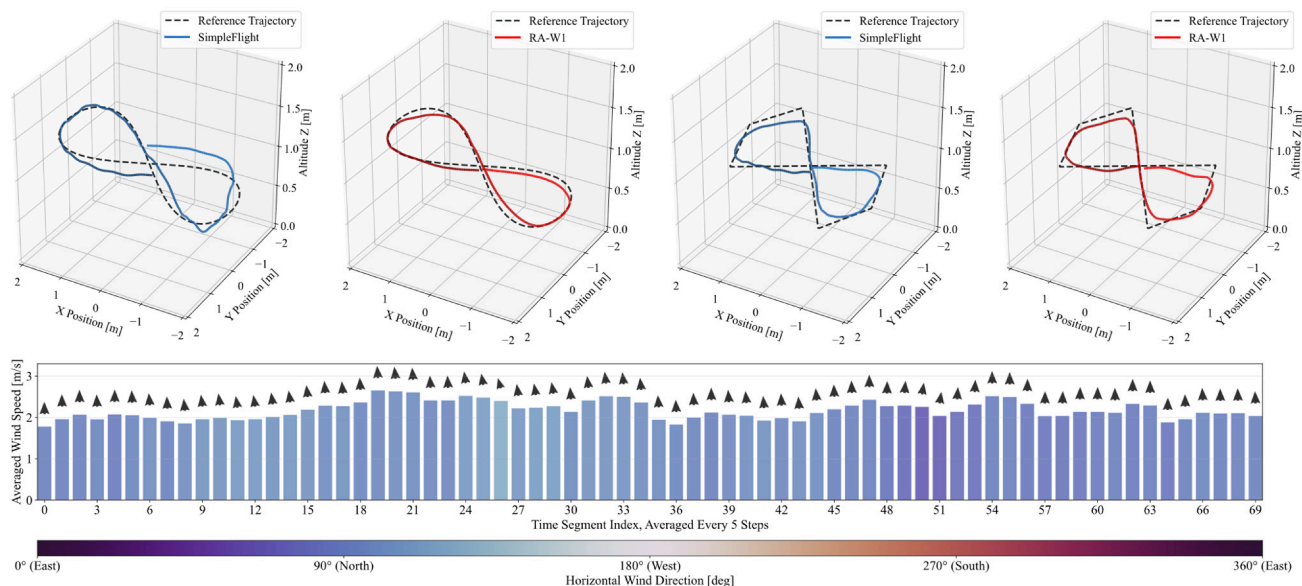
**Figure 8.** Representative qualitative trajectory-tracking examples under measured Gulf of Eilat wind sequences. The lower panel visualizes the five-step averaged wind speed, and the color bar indicates the horizontal wind direction.

Table 7 shows that RA-W1 brings consistent improvements under the measured real-wind replay. On the smooth lemniscate, RA-W1 reduces Avg. Error from 0.324 to 0.224 (30.9%), E_{95} Error from 0.807 to 0.587 (27.3%), and CVaR_{0.9} Error from 0.855 to 0.684 (20.0%), and raises PCR from 97.41% to 100.00%. On the pointed figure-eight, RA-W1 reduces Avg. Error from 0.301 to 0.245 (18.6%), E_{95} Error from 0.687 to 0.585 (14.8%), and CVaR_{0.9} Error from 0.735 to 0.626 (14.8%), with PCR rising from 83.78% to 99.25%. These results suggest that the compact residual feature remains useful under measured environmental wind, even when the improvement margins are smaller than those observed on the synthetic wind sets. The largest practical gain appears in progress completion on the sharp-corner trajectory, where PCR improves by 15.47 percentage points.

6. Conclusions

This paper presented a residual-aware DRL framework for robust quadrotor trajectory tracking under multimodal wind disturbances. The method augments a quadrotor trajectory-tracking task with a multimodal wind disturbance pool, an online acceleration-residual observation channel, residual-input clipping and dropout, and a reward that combines position tracking, action smoothness, acceleration smoothness, and a CVaR-style tail-risk penalty. The framework was evaluated on a smooth lemniscate and a pointed figure-eight trajectory under in-distribution wind, held-out out-of-distribution wind, and measured real-wind disturbances.

The main finding is that compact residual information is useful, but that more residual memory is not necessarily better. Across both trajectory geometries and across all

three wind conditions, the one-step residual variant delivered the most balanced behavior: it improved not only the average tracking accuracy but also the upper tail of the error distribution and the completion of the reference trajectory, and it did so consistently rather than by excelling on any single metric. The long-history variant behaved in the opposite way. It converged fastest during training and reached the lowest training error, yet it lost that advantage as soon as the wind distribution shifted, and its held-out tracking degraded severely.

The ablation study indicates that the residual observation and the tail-risk penalty are complementary rather than redundant. Residual observation alone improved mean tracking accuracy but did not reliably suppress the error tail, particularly once the wind distribution shifted, whereas tail-risk shaping alone suppressed large deviations without providing the disturbance awareness needed to track sharp reference transitions. Only their combination improved mean accuracy, tail behavior, and trajectory completion at the same time. The measured real-wind replay qualitatively supports the same picture: the compact residual feature remained beneficial on wind that was recorded rather than generated, with the clearest practical gain appearing in the completion of the sharp-corner trajectory.

These results also highlight the scope and limitations of the present study. The residual-window sensitivity study showed that, under the tested settings, longer histories accelerated training but did not improve held-out robustness; $K = 1$ provided the best overall balance between disturbance awareness and generalization. The study remains simulation-based; measured real-wind replay improves disturbance realism, but hardware experiments and more detailed modeling of actuator delay, sensor noise, and aerodynamic uncertainty are still needed.

Author Contributions: Conceptualization, K.Q. and Z.L.; methodology, K.Q.; software, K.Q.; validation, K.Q.; formal analysis, K.Q. and C.L.; investigation, K.Q. and C.L.; resources, Z.L.; data curation, K.Q.; writing—original draft preparation, K.Q.; writing—review and editing, Z.L.; visualization, K.Q.; supervision, Z.L.; project administration, K.Q.; funding acquisition, Z.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was funded by the National Natural Science Foundation of China, grant number 62576355.

Data Availability Statement: The original data presented in this study are openly available in Zenodo at <https://doi.org/10.5281/zenodo.20812941>. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations and mathematical symbols are used in this manuscript:

CTBR	Collective thrust and body rate
CVaR	Conditional value-at-risk
DRL	Deep reinforcement learning
ID	In-distribution
MDP	Markov decision process
OOD	Out-of-distribution
PCR	Progress completion ratio
PID	Proportional-integral-derivative
PPO	Proximal policy optimization

RA-WK	Residual-aware policy with a residual observation window of K steps; K denotes the residual-window length
RL	Reinforcement learning
SE(3)	Special Euclidean group of rigid-body poses
SO(3)	Special orthogonal group of three-dimensional rotation matrices
UAV	Unmanned aerial vehicle
$\mathbf{p}, \mathbf{v}$	Inertial-frame position and linear velocity of the quadrotor (m, m/s)
$\mathbf{R}, \boldsymbol{\omega}$	Attitude rotation matrix and body-frame angular velocity (-, rad/s)
$m, \mathbf{J}, \mathbf{g}$	Vehicle mass, inertia matrix, and gravity vector (kg, kg m ² , m/s ²)
$\mathbf{f}^B, \boldsymbol{\tau}$	Body-frame collective thrust vector and control torque (N, N m)
$\mathbf{f}_w, \boldsymbol{\tau}_w, \mathbf{f}_{aero}$	Wind disturbance force, wind disturbance torque, and aerodynamic residual force (N, N m, N)
$\mathbf{w}_t, e_t$	Realized three-axis wind vector and instantaneous Euclidean tracking error (m/s, m)
$\mathbf{a}_t^{\text{meas}}, \mathbf{a}_t^{\text{nom}}$	Measured and nominal translational accelerations (m/s ²)
k_d	Nominal quadratic-drag coefficient of the residual generator (kg/m)
$\mathbf{d}_t, c_d, \mathbf{z}_t^d$	Clipped acceleration residual, residual clipping bound, and flattened residual feature
p_{drop}	Probability of masking the residual feature during training
$\mathbf{o}_t, \mathbf{u}_t$	Policy observation and normalized CTBR command
$T_{\text{des}}, \boldsymbol{\omega}_{\text{des}}$	Desired collective thrust and desired body rates (N, rad/s)
$\mathbf{M}, l, c_\tau$	Control-allocation matrix, arm length, and torque-to-thrust ratio
$e_{\text{CVaR}}, \lambda_{\text{CVaR}}$	Tail-risk error threshold and tail-risk penalty weight
$\odot, \ \cdot\ _2, \text{clip}(\cdot)$	Element-wise product, Euclidean norm, and element-wise saturation operator

References

1. Mohsan, S.A.H.; Khan, M.A.; Noor, F.; Ullah, I.; Alsharif, M.H. Towards the Unmanned Aerial Vehicles (UAVs): A Comprehensive Review. *Drones* **2022**, *6*, 147. [\[CrossRef\]](#)
2. Mellinger, D.; Kumar, V. Minimum Snap Trajectory Generation and Control for Quadrotors. In Proceedings of the 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 9–13 May 2011; pp. 2520–2525. [\[CrossRef\]](#)
3. Lee, T.; Leok, M.; McClamroch, N.H. Geometric Tracking Control of a Quadrotor UAV on SE(3). In Proceedings of the 49th IEEE Conference on Decision and Control, Atlanta, GA, USA, 15–17 December 2010; pp. 5420–5425. [\[CrossRef\]](#)
4. Mahony, R.; Kumar, V.; Corke, P. Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor. *IEEE Robot. Autom. Mag.* **2012**, *19*, 20–32. [\[CrossRef\]](#)
5. Lopez-Sanchez, I.; Moreno-Valenzuela, J. PID Control of Quadrotor UAVs: A Survey. *Annu. Rev. Control* **2023**, *56*, 100900. [\[CrossRef\]](#)
6. Guo, Y.; Sun, Q.; Pan, Q. Robust Tracking Control of Underactuated UAVs Based on Zero-Sum Differential Games. *Drones* **2025**, *9*, 477. [\[CrossRef\]](#)
7. Wang, J.; Alattas, K.A.; Bouteraa, Y.; Mofid, O.; Mobayen, S. Adaptive Finite-Time Backstepping Control Tracker for Quadrotor UAV with Model Uncertainty and External Disturbance. *Aerosp. Sci. Technol.* **2023**, *133*, 108088. [\[CrossRef\]](#)
8. Wang, R.; Shen, J. Disturbance Observer and Adaptive Control for Disturbance Rejection of Quadrotor: A Survey. *Actuators* **2024**, *13*, 217. [\[CrossRef\]](#)
9. Ramirez-Neria, M.; Luviano-Juárez, A.; González-Sierra, J.; Ramírez-Juárez, R.; Aguerrebere, J.; Hernan-dez-Martinez, E.G. Active Disturbance Rejection Control for the Trajectory Tracking of a Quadrotor. *Actuators* **2024**, *13*, 340. [\[CrossRef\]](#)
10. Xing, Z.; Zhang, Y.; Su, C.-Y. Active Wind Rejection Control for a Quadrotor UAV Against Unknown Winds. *IEEE Trans. Aerosp. Electron. Syst.* **2023**, *59*, 8956–8968. [\[CrossRef\]](#)
11. Bauersfeld, L.; Kaufmann, E.; Foehn, P.; Sun, S.; Scaramuzza, D. NeuroBEM: Hybrid Aerodynamic Quadrotor Model. In Proceedings of the Robotics: Science and Systems XVII, Virtual Event, 12–16 July 2021. [\[CrossRef\]](#)
12. Torrente, G.; Kaufmann, E.; Foehn, P.; Scaramuzza, D. Data-Driven MPC for Quadrotors. *IEEE Robot. Autom. Lett.* **2021**, *6*, 3769–3776. [\[CrossRef\]](#)
13. Das, A.; Yadav, R.D.; Sun, S.; Sun, M.; Kaski, S.; Pan, W. DroneDiffusion: Robust Quadrotor Dynamics Learning with Diffusion Models. In Proceedings of the 2025 IEEE International Conference on Robotics and Automation (ICRA), Atlanta, GA, USA, 19–23 May 2025; pp. 1604–1610. [\[CrossRef\]](#)

14. Ren, Y.; Zhu, F.; Sui, S.; Yi, Z.; Chen, K. Enhancing Quadrotor Control Robustness with Multi-Proportional-Integral-Derivative Self-Attention-Guided Deep Reinforcement Learning. *Drones* **2024**, *8*, 315. [\[CrossRef\]](#)
15. Huang, K.; Rana, R.; Spitzer, A.; Shi, G.; Boots, B. DATT: Deep Adaptive Trajectory Tracking for Quadrotor Control. In *Proceedings of the 7th Conference on Robot Learning, Atlanta, GA, USA, 6–9 November 2023*; Proceedings of Machine Learning Research; PMLR: Cambridge, MA, USA, 2023; Volume 229, pp. 326–340.
16. Eschmann, J.; Albani, D.; Loianno, G. Learning to Fly in Seconds. *IEEE Robot. Autom. Lett.* **2024**, *9*, 6336–6343. [\[CrossRef\]](#)
17. Ma, Q.; Wu, Y.; Shoukat, M.U.; Yan, Y.; Wang, J.; Yang, L.; Yan, F.; Yan, L. Deep Reinforcement Learning-Based Wind Disturbance Rejection Control Strategy for UAV. *Drones* **2024**, *8*, 632. [\[CrossRef\]](#)
18. Loquercio, A.; Kaufmann, E.; Ranftl, R.; Dosovitskiy, A.; Koltun, V.; Scaramuzza, D. Deep Drone Racing: From Simulation to Reality with Domain Randomization. *IEEE Trans. Robot.* **2020**, *36*, 1–14. [\[CrossRef\]](#)
19. Loquercio, A.; Kaufmann, E.; Ranftl, R.; Müller, M.; Koltun, V.; Scaramuzza, D. Learning High-Speed Flight in the Wild. *Sci. Robot.* **2021**, *6*, eabg5810. [\[CrossRef\]](#) [\[PubMed\]](#)
20. Kaufmann, E.; Bauersfeld, L.; Loquercio, A.; Müller, M.; Koltun, V.; Scaramuzza, D. Champion-Level Drone Racing Using Deep Reinforcement Learning. *Nature* **2023**, *620*, 982–987. [\[CrossRef\]](#) [\[PubMed\]](#)
21. Rockafellar, R.T.; Uryasev, S. Optimization of Conditional Value-at-Risk. *J. Risk* **2000**, *2*, 21–42. [\[CrossRef\]](#)
22. Chow, Y.; Tamar, A.; Mannor, S.; Pavone, M. Risk-Sensitive and Robust Decision-Making: A CVaR Optimization Approach. In *Proceedings of the 29th Conference on Neural Information Processing Systems, Montreal, QC, Canada, 7–12 December 2015*; pp. 1522–1530.
23. Bellemare, M.G.; Dabney, W.; Munos, R. A Distributional Perspective on Reinforcement Learning. In *Proceedings of the 34th International Conference on Machine Learning, Sydney, Australia, 6–11 August 2017*; pp. 449–458.
24. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.
25. Song, Y.; Romero, A.; Müller, M.; Koltun, V.; Scaramuzza, D. Reaching the Limit in Autonomous Racing: Optimal Control versus Reinforcement Learning. *Sci. Robot.* **2023**, *8*, eadg1462. [\[CrossRef\]](#) [\[PubMed\]](#)
26. Chen, J.; Yu, C.; Xie, Y.; Gao, F.; Chen, Y.; Yu, S.; Tang, W.; Ji, S.; Mu, M.; Wu, Y.; et al. What Matters in Learning a Zero-Shot Sim-to-Real RL Policy for Quadrotor Control? A Comprehensive Study. *IEEE Robot. Autom. Lett.* **2025**, *10*, 7134–7141. [\[CrossRef\]](#)
27. Tobin, J.; Fong, R.; Ray, A.; Schneider, J.; Zaremba, W.; Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, Vancouver, BC, Canada, 24–28 September 2017*; pp. 23–30. [\[CrossRef\]](#)
28. Geirhos, R.; Jacobsen, J.H.; Michaelis, C.; Zemel, R.; Brendel, W.; Bethge, M.; Wichmann, F.A. Shortcut Learning in Deep Neural Networks. *Nat. Mach. Intell.* **2020**, *2*, 665–673. [\[CrossRef\]](#)
29. Xu, B.; Gao, F.; Yu, C.; Zhang, R.; Wu, Y.; Wang, Y. OmniDrones: An Efficient and Flexible Platform for Reinforcement Learning in Drone Control. *IEEE Robot. Autom. Lett.* **2024**, *9*, 2838–2844. [\[CrossRef\]](#)
30. Meier, L.; Honegger, D.; Pollefeys, M. PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms. In *Proceedings of the 2015 IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 26–30 May 2015*; pp. 6235–6240. [\[CrossRef\]](#)
31. Shani-Zerbib, A.; Liberzon, D. *Data Set of Wind-Waves Interactions in the Gulf of Eilat (IUI), June 2017*; Mendeley Data: London, UK, 2018; Version 2. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.